

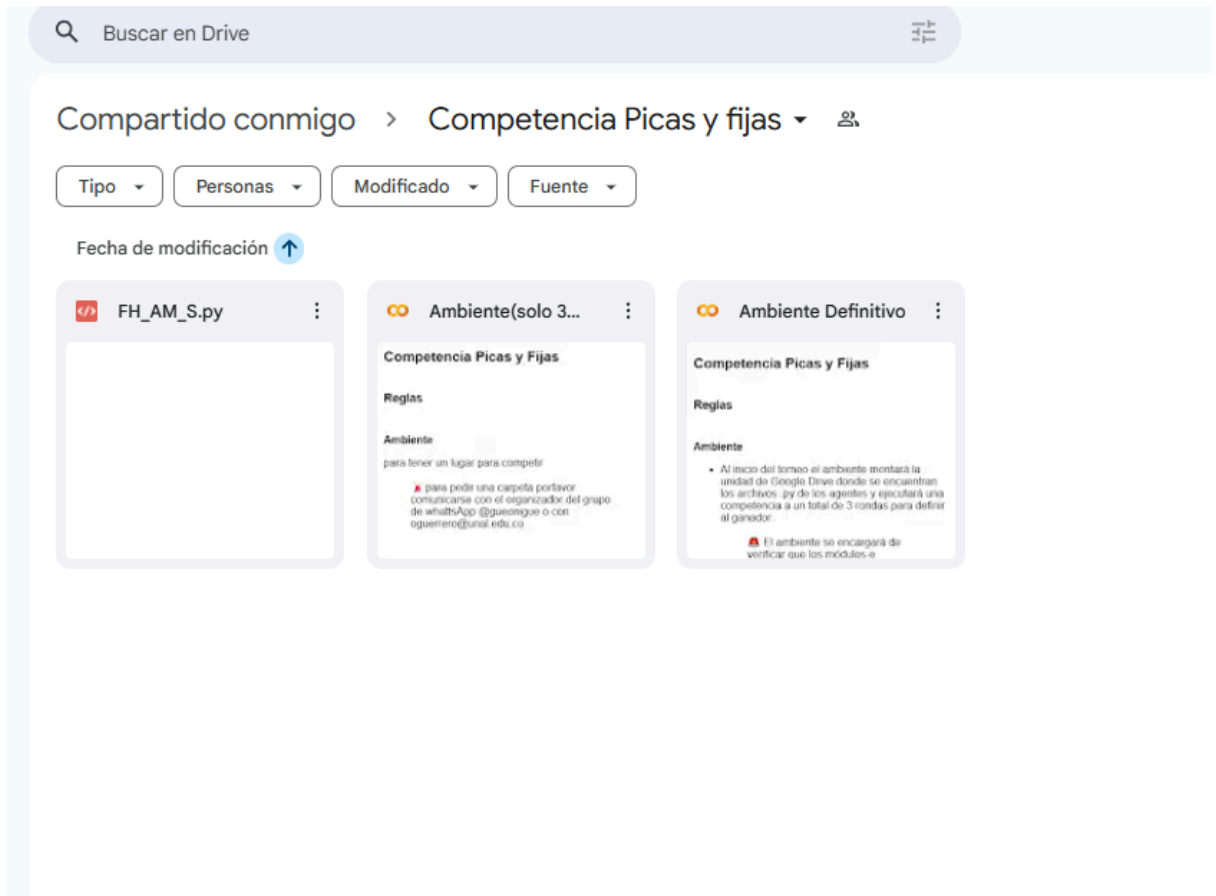
Torneo oficial de picas y fijas - Clase de introducción a sistemas inteligentes

REGLAS

1. Se programará en Python usando Google Drive.
2. Se compite bajo la supervisión de un juez virtual centralizado.
3. El objetivo principal es adivinar un número secreto de 4 dígitos únicos.
4. El juez es el único encargado de evaluar los intentos y devolver las picas y fijas.
5. El criterio para ganar una ronda es ser el primero en lograr 4 fijas [0, 4].
6. Si un intento tiene un formato inválido, se penaliza perdiendo ese turno.
7. Si ambos agentes adivinan el número en el mismo turno, la ronda termina en empate.
8. El ganador del torneo se define por quien acumule más victorias al finalizar todas las rondas configuradas.
9. El archivo de colab “ambiente definitivo” el código y sus reglas se usarán para hacer el torneo
10. Se debe de formatear el agente en base a la interfaz en el archivo de google colab
11. El nombre de la clase de su agente debe de tener un nombre identificable
12. El agente de cada equipo debe de subirse a la siguiente carpeta en drive:
<https://drive.google.com/drive/folders/1yeMyvkuwyy6DYz0H5S2LN1VWqs5NcmyW>
13. El “modo torneo” es donde se define cada partida, mientras que el otro modo es para verificar la función del agente.
14. Se permite modificaciones al código en el torneo mientras solo sea para acomodar el formato
15. Todos los agentes deben de subirse en clase antes del inicio del torneo para hacer el bracket y tener constancia del código original de cada uno

*Se sugiere que al correr el código de ambiente se verifique que su agente aparece

1. Entrar al archivo de ambiente definitivo



2. Seleccionar logo de carpeta

CO Ambiente Definitivo ☆ ☁

Fichier Modifier Affichage Insérer Exécution Outils Aide

Q Commandes + Code + Texte ▶ Tout exécuter ▾

MessageError: Error: credential propagation was unsuccessful

▼ Ambiente(código de ambiente juez)

[4] ✓ 0 s

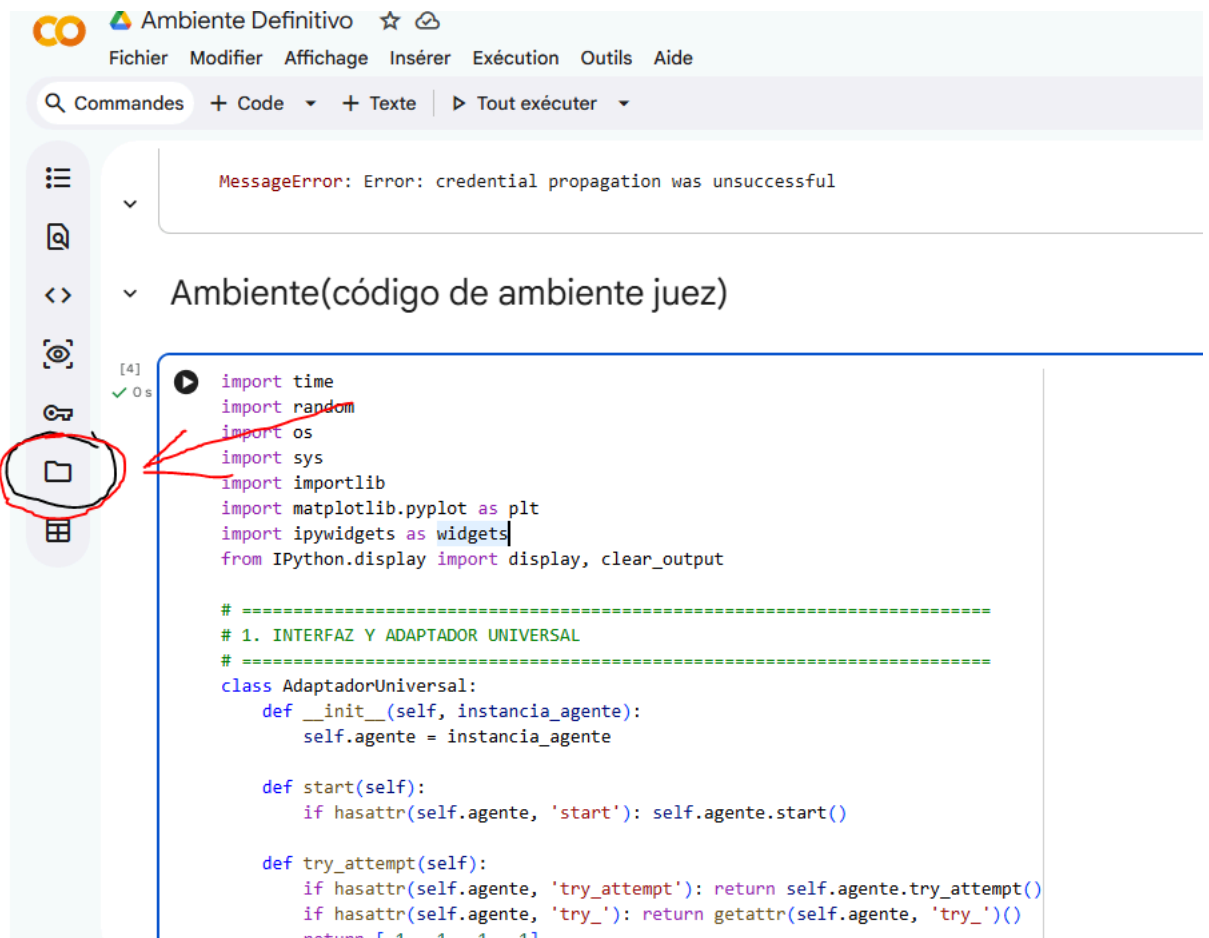
```
import time
import random
import os
import sys
import importlib
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output

# =====
# 1. INTERFAZ Y ADAPTADOR UNIVERSAL
# =====

class AdaptadorUniversal:
    def __init__(self, instancia_agente):
        self.agente = instancia_agente

    def start(self):
        if hasattr(self.agente, 'start'): self.agente.start()

    def try_attempt(self):
        if hasattr(self.agente, 'try_attempt'): return self.agente.try_attempt()
        if hasattr(self.agente, 'try_'): return getattr(self.agente, 'try_')()
        return [1, 1, 1, 1]
```

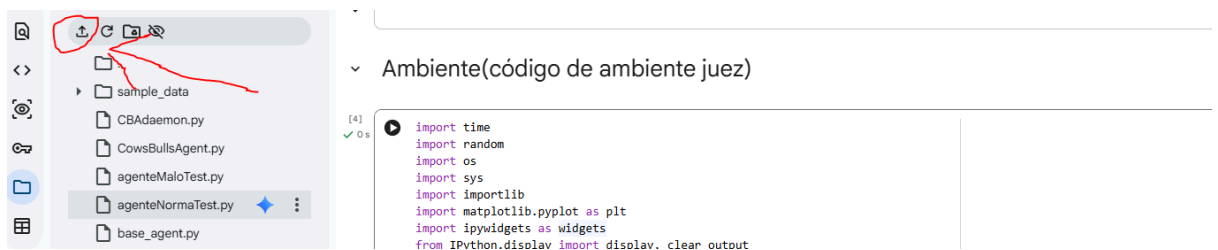


3. Subir archivo de agente

▼ Ambiente(código de ambiente juez)

[4] ✓ 0 s

```
import time
import random
import os
import sys
import importlib
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output
```



4. Luego se corre el código del ambiente

▼ Ambiente(código de ambiente juez)

```
[4]
✓ 0 s
import time
import random
import os
import sys
import importlib
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output

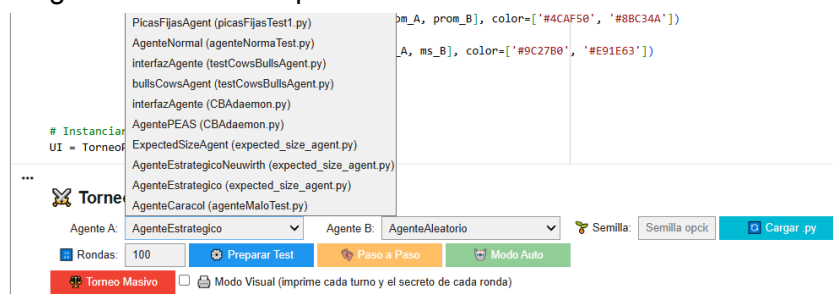
# =====
# 1. INTERFAZ Y ADAPTADOR UNIVERSAL
# =====

class AdaptadorUniversal:
    def __init__(self, instancia_agente):
        self.agente = instancia_agente

    def start(self):
        if hasattr(self.agente, 'start'): self.agente.start()

    def try_attempt(self):
        if hasattr(self.agente, 'try_attempt'): return self.agente.try_attempt()
        if hasattr(self.agente, 'try_'): return getattr(self.agente, 'try_')()
```

5. Su agente debería de aparecer en el menú



6. Se selecciona la cantidad de rondas y si se quiere hacer test o torneo(y si se quiere que se muestre turno por turno o que sea un background process).

*Los agentes “agente inteligente” y “agente aleatorio” son por defectos del código